ParityKeeper v1.2 — Verified Code Audit, Fixes and Developer Notes

Release: 1.2.0, 19 October 2026
Applies to: Windows (MultiPar `par2j64.exe`) and Debian Linux (`par2cmdline`, `par2`)
Primary inputs: the user-supplied *ParityKeeper v1.1 — Code Analysis Report*, actual v1.1 Windows and Debian source ZIPs, the previous 45-page v1.1 technical specification, and the newly revised v1.2 source.
Read before production use: v1.2 was tested with *simulated* PAR2 engines; real MultiPar / `par2` must still pass the application's disposable self-test on the intended computer. Keep both physical HDD copies. Do not test destructive recovery on your only surviving copy.

1. Scope, methodology, evidence

This review compared findings to *both* v1.1 branches, rather than assuming identical source files. This matters: the report calls out some real vulnerabilities, but also includes inaccurate claims. We also checked the official engine manuals for PAR2 exit statuses and command-line CPU options. Fixes were introduced conservatively, with failure modes that preserve the source files and committed recovery data. A new regression suite exercises filenames differing only by case, migration, content races, missing mirrors, path traversal, backup quarantine collisions and platform-specific exit handling.

Important conclusion: the report's statement that the application was safe for production is **not an independent certification**. v1.1 included material edge cases. v1.2 improves them, but is not yet field-validated with the real Windows and Debian PAR2 binaries, 7 TB-scale data sets, or actual hard-drive failure scenarios.

2. Issue-by-issue disposition

Report finding	Verification against v1.1	v1.2 corrective action / status
Linux filenames collide under `NOCASE`	**Confirmed.** `files` and temporary `seen` used `COLLATE NOCASE`. SQLite usually raises a uniqueness error rather than silently overwriting a duplicate; either behavior prevents tracking both.	**Fixed.** Both tables use `BINARY`. Existing databases migrate transactionally on normal opening, with a full SQLite copy kept under snapshots before conversion. All group-matching logic now uses exact path strings, not `.casefold()`.
Missing-file deletion dialog unclear	**Confirmed usability issue.** `apply_changes()` already refused reappeared files and did not delete filesystem sources.	**Fixed.** Approval prompt expressly says **inventory entry only**; source files are already missing and are not deleted by the program.
Quarantine destination collision and ignored failures	**Confirmed.** Quarantine name was based on second-level timestamp; `shutil.move()` failure was logged and swallowed.	**Fixed.** Unique timestamp+UUID directory, no overwrite on collision, explicit returned errors. On incomplete quarantine, the UI retains the repair issue for manual inspection even when restored hashes are good.
Mirror missing-inventory validation	**Confirmed.** A missing folder inventory produced a less useful exception.	**Fixed.** Both inventories are checked in the engine and GUI; comparing does not create a missing database.

Report finding	Verification against v1.1	v1.2 corrective action / status
File changes during PAR creation despite stable size/mtime	**Confirmed limited race.** Code hashed before PAR generation but checked only size/mtime afterward.	**Fixed.** A second streaming SHA256+CRC32 comparison for **every** member is required after PAR creation and external verification, before the SQLite commit. Costs an additional complete source read; progress estimate updated.
Same PAR2 exit-code masks used on Linux and Windows	**Partly incorrect report.** Debian v1.1 already handled numeric 0/1 verify and 0 repair separately. Windows used bit flags.	**Clarified and hardened.** Added named per-platform success/repairability helpers and regression tests. Never evaluate Linux numeric codes using Windows bitmasks. See section 5.
`live_threads` GUI/worker synchronization	**Confirmed implementation gap**, though individual Python dict reads are usually atomic under CPython.	**Fixed.** `threading.Lock` guards both slider writes and worker reads. Critically, the lock is acquired **inside** the worker's provider callback—not only while the callback is created.
No persistent activity log	**Confirmed.** Only the Tk text widget retained messages.	**Fixed.** `activity.log` with 5 MB rotating files and 3 backups is kept in the user's ParityKeeper configuration directory. It captures GUI logs and PAR2 messages delivered through the GUI.

Report finding	Verification against v1.1	v1.2 corrective action / status
Orphan PAR directories accumulate after interruption	**Confirmed.** Uncommitted recovery folders can remain if the process crashes.	**Safely addressed.** Added **Review leftover PAR2** button and `list_orphan_par_dirs`. **No automatic deletion**, because unfinished PAR files may still aid recovery.
No persistent resume state	**Partly valid.** v1.1 already commits per successful group and resumes by rescanning, but did not record interruption state.	**Partially addressed.** `last_apply_state.json` records state and committed group count. It is *diagnostic only*: re-scan and re-approve outstanding changes. Never replay old approvals automatically.
Path traversal handling too basic	**Partial concern.** `../` and absolute paths were already rejected; URL percent encodings are not decoded into filesystem traversal. The report's `str(p).startswith(str(root))` replacement is itself unsafe (`/data/a` vs `/data/another`).	**Hardened.** Reject non-canonical/absolute/parent paths, NULs, symlink/junction traversal and paths escaping the canonical root. Internal PAR references have separate allowlisting and UUID validation.
Memory grows with very large result lists	**Confirmed architectural limitation.** GUI pages the view, but retains all results in Python lists.	**Not fully fixed.** SQLite-backed lazy result pagination remains future work. Affected mostly by **file count**, not total terabytes.

Report finding	Verification against v1.1	v1.2 corrective action / status
Report: "comparison has `@exclusive_archive`"	**Incorrect** for supplied Windows and Debian v1.1 source: the function has no decorator. The original v1.1 specification saying it lacks an archive lock was accurate.	**No behavioral change.** Comparison remains baseline-only and read-oriented; simultaneous inventory writers are not excluded across both roots. Use comparison with no other ParityKeeper job running on either drive.
Report's overall count of 19 issues	**Not itemized in full.** The text lists multiple substantive findings but not individually demonstrable evidence for every count.	We address listed verifiable findings above; we do not claim 19 uniquely fixed bugs.

3. File layout, inventory, and migration

An archive stores its own `.paritykeeper/` directory:

```
<backup-root>/
  <user photos, text, videos, folders ...>
  .paritykeeper/
    inventory.sqlite3
    operation.lock
    last_apply_state.json               # NEW: diagnostic checkpoint
    par2/<UUID>/data.par2
    par2/<UUID>/*.vol*.par2
    snapshots/inventory_*.sqlite3
    snapshots/inventory_before_case_migration_<UUID>.sqlite3  # NEW: retained
    quarantine/<timestamp>_<UUID>/...    # NEW: collision-resistant path
```

The SQLite `sets` and `events` tables are unchanged. `files` retains the same columns and values, but the primary key is now `path TEXT PRIMARY KEY COLLATE BINARY`. The temporary scan table `seen` uses `BINARY`, and SQL joins use exact case-sensitive path values. Cross-platform parity member updates, deletions and grouping must not use `.casefold()` to select inventory rows.

Migration algorithm: with the archive lock held by Scan/Apply, open a legacy `NOCASE` inventory; create a separate complete SQLite backup with SQLite's `backup()` API; start a transaction; copy rows into a newly created `files_case_sensitive` table with the identical columns but BINARY PK; drop the old table; rename the new table; rebuild `files_by_set`; commit. An

error rolls back the transaction and reports the backup path. The snapshot copy is intentionally **not** managed by the normal three-snapshot rotation. Scanning with v1.2 detects missing case variants as NEW and they can be approved/protected normally.

Safety caveat: If an earlier program somehow lost an inventory entry, migration cannot reconstruct its old checksum; scan will discover the file as NEW. Existing PAR sets on disk remain intact. Always retain a physically separate mirror before upgrading.

4. Exact PAR-set update transaction and TOCTOU mitigation

For each small group or single large file:

1. Validate selected operations. `NEW` is selected by default; `CHANGED` and `MISSING` are initially unchecked. A missing file must still be absent when Apply starts.
2. Hash incoming files using 8 MiB chunks, producing `(size, mtime_ns, SHA256, CRC32)`.
3. For retained members in an old set, hash and verify content against the current approved baseline before including those members in a new PAR set.
4. Build recovery files in a fresh UUID directory. Engines are invoked directly with a **list of arguments**, not via shell command interpolation.
5. Require successful PAR2 creation and at least one recovery volume for nonempty source data.
6. Check each member's size/mtime for ordinary writes detected while creating the parity files.
7. Run the native PAR2 **verify** command. Only a success result is allowed for the newly created set.
8. **NEW IN 1.2:** read and hash **every member again**. Fail if size, SHA256 or CRC32 differs from the hash captured before creation, even if mtime and size were deliberately restored.
9. In a single SQLite transaction, add the new verified set, redirect the file records, retire the old set, and log the event. Commit.
10. Only **after commit** may the implementation retire/clean the superseded PAR2 directory. A failed pre-commit operation leaves the old approved baseline intact.
11. Persist progress (`groups_committed`, `updated_utc`, `requires_new_scan`) to `last_apply_state.json`. This is not an authorization to resume without rechecking content.

This defense materially reduces a time-of-check/time-of-use race, but no application-level hashing can provide atomic filesystem snapshots while other software concurrently writes the same files. **Do not modify source files during PAR creation, validation or repair.** Keep your second physical

copy.

5. Platform-specific PAR2 command semantics

Windows MultiPar (`par2j64.exe`)

Creation uses `c`, `/fu` UTF-8 file list, `/d<root>`, `/uo`, `/lcN`, and `/rrP` (large) or `/ssSIZE` plus `/rnBLOCKS` (small). Verification uses `v /lcN /d<root> /uo <index>`. Repair uses `r /b /lcN /d<root> /uo <index>`; `/b` preserves damaged originals as numbered siblings when generated. `/lc1` through `/lc10` really does limit the number of processing threads, per MultiPar's detailed documentation.

Windows MultiPar return status is bitwise: 0 normal, bit 1 fatal, 2 canceled, 4 incomplete sources, 8 insufficient repair blocks, 16 repair succeeded, 256 incomplete parity; 32/64/128 indicate candidate recovery strategies when coupled with incomplete-source flags. Verification may return 132 (=128+4) when repair is possible; do **not** treat all nonzero verify codes as fatal. Repair command results `0` and `16` are treated as acceptable for proceeding to post-repair SHA256/CRC32, but failure flags (1/2/4/8/256) are rejected. **A nominally successful engine return is not proof of restored content**: hashes of all set members must match.

Debian (`par2` / par2cmdline)

Creation uses `c -tN -Tmin(N,2) -B<root> <index> -- <relative filenames>`. Verification uses `v -tN -Tmin(N,2) -B<root> <index>`; repair `r` with the same options. A single subprocess handles each set. Existing v1.1 Debian code already used the numeric return classes correctly.

Debian exit statuses are enumerated, not bitwise: `0` complete, `1` repair possible (verify), `2` not repairable, `3` invalid flags, `4` missing critical data, `5` failed repair, `6` IO error, `7` internal error, `8` memory error. Run repair only when verification returns `0` or `1`; accept a repair result only if `0`, then independently check all member hashes.

The user-adjustable 1–10 small/large **CPU threads** are engine-internal processing threads; this program does not run 10 independent parity tasks. Slider writes and future subprocess thread-limit reads are synchronized with `threading.Lock`. A new slider value takes effect at the next subprocess, not midway through one.

Compatibility caution: Linux and Windows use portable PAR2 format, but differing filename case, separators, and Unicode handling can still make old sets incompatible. Verify a *copy of a small Windows-generated PAR set* using native Linux `par2` before updating an archive from the other OS. Do not concurrently manage a single archive with two installations.

6. Corrupt-file repair and quarantine protocol

- CRC Check reads all tracked bytes and compares SHA256 and CRC32; it reports corruption, missing or unreadable files and PAR recovery issues. A metadata-only Scan does not detect unchanged-metadata bitrot.
- The user selects file issues. Within each recovery set, all damaged members must be selected before repair is attempted; a single repair command may reconstruct several files.
- Before repair, take a directory-name inventory of preexisting `<name>.<number>` damaged-original backups. If a parent directory cannot be enumerated, stop rather than pretending no copies exist.
- Run native verify to assess recoverability. If repair is attempted, retain damaged-original copies according to engine behavior (MultiPar `/b` on Windows; native `par2` behavior on Debian).
- Move **newly generated** numbered damaged-original backups into a fresh quarantine folder using timestamp+UUID and a collision-safe destination. Existing similarly named files are not moved.
- Quarantine errors are explicitly logged. After repair, hash all set members against the approved inventory. Only when hashes all match **and** all newly produced backups were quarantined successfully is the issue marked resolved automatically.
- If quarantine failed, keep the issue in the UI for manual review. Recovered source contents may still be correct; the application must not disguise a cleanup error as unqualified success.

No code in this release silently deletes files intentionally removed by the user; deletion approval changes inventory membership and regenerates parity for survivors. No auto-pruning of orphan PAR2 folders occurs.

7. Persistent log, checkpoints, remaining limitations

GUI activity is retained in `activity.log` (rotate at 5 MB, keep 3 numbered archives):

- Windows: `%APPDATA%\\ParityKeeper\\activity.log`
- Debian: `${XDG_CONFIG_HOME:-~/.config}/paritykeeper/activity.log`

`last_apply_state.json` in each archive root records `running`, `interrupted; scan again to continue safely`, or `completed`, together with a committed group count and the latest UTC timestamp. It is written by atomic rename on a best-effort basis; **never** replay prior user selections from this file. A subsequent metadata Scan identifies unprotected new files and requires approval for changes/deletions.

`Review leftover PAR2` shows directories under `.paritykeeper/par2/` without any matching set entry in SQLite. It **never automatically deletes or renames them**. Recoverable intermediate files can survive crashes. Orphans with stale *retired* database rows will not be reported, and may still use disk space.

Any eventual cleanup command should require source hashes, index verification, explicit selection, and user confirmation.

Not yet implemented / not guaranteed: GUI does not use lazy SQLite-backed pagination of all scan results; millions of files may use a lot of RAM. No unattended/automatic resumption without a fresh scan. No protection against deliberate tampering with the SQLite baseline or hardware-level simultaneous corruption on both HDDs. No real disk throughput/time benchmark was run in these tests. No automatic filesystem snapshot when other applications are writing. Mirror compare checks **stored inventory values**, not live mirror contents, and does not atomically lock both archives.

8. Regression tests and reconstruction

As-tested: Windows v1.2: **17/17** simulated-engine tests pass; Debian v1.2: **20/20** simulated-engine tests pass. GUI smoke checks on both passed in a virtual X display and verified thread sliders, new button, and persistent logging. Automation used a *fake PAR2 executable*, which simulates recovery and stores test bytes; its test results do NOT validate the real engine's recovery power.

Run Windows source tests from its source directory:

```
py -m unittest discover -s tests -v
```

Run Debian tests from source root:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

Always click **Test MultiPar** (Windows) or **Test PAR2** (Debian) after installation. Then test an isolated folder and deliberately corrupt **copies** of sample files before allowing application work on your main 7 TB archive. Keep at least one independently verified copy of the `.paritykeeper` database and recovery sets, as well as user data.

New v1.2 test cases

1. Scan/protect `Photo.jpg` and `photo.jpg` as separate Linux/Windows test files.
2. Open an old `NOCASE` inventory, migrate atomically and confirm an independent pre-migration SQLite snapshot exists.
3. Attempt mirror comparison when the other root has no inventory; fail without creating one.
4. Reject relative `../`, malformed and metadata-source paths; block symlink traversal outside the archive.
5. Quarantine two numbered copies with the same relative name; verify neither is overwritten.
6. Simulate modification while PAR creation runs, restore original file size/mtime, and ensure no baseline is committed even if the fake PAR verification reports success.

7. Check orphan directory listing leaves all data in place.
8. Test MultiPar flag combinations and Linux numeric status classes.
9. Run original v1.1 end-to-end regression cases for adds, edits, approved deletions, hash detection, recovery and saved-inventory comparisons.
10. Smoke test GUI launch and persistent log creation on both versions.

9. Precautions for the two-HDD 7 TB deployment

First test v1.2 against a throwaway folder, including an old v1.1 inventory migration on a **copy**, so you can verify that Linux case-sensitive entries are correctly discovered. Before opening either real HDD archive in v1.2, separately copy the entire `.paritykeeper/` folder to the other disk or a third location; free capacity matters, and the recovery files may be large. Prefer maintaining the two existing physical copies independently, and do not run simultaneous Apply/Repair operations against the same archive. Use Scan to identify new/changed/deleted files, never approve suspicious differences, and use CRC Check periodically to detect silent content damage. Use the engine's disposable self-test to validate real recovery before relying on it.

10. External technical references

- MultiPar command-line manual: options, `/lc` and exit-bit definitions
- par2cmdline upstream documentation
- par2cmdline C enum declarations, Debian source

This document is part of the v1.2 source release. Refer to `engine.py`, `gui.py`, `progress.py`, `tests/`, plus the separate original v1.1 Technical Specification included as historical reference for the full GUI/API reconstruction catalog. The verified changes and cautions in this v1.2 document take precedence wherever that historical specification differs.